Álgebra Lineal Computacional .UBA EXACTAS

2do cuatrimestre 2026

Trabajo Práctico: Calibración y Pose de la Cámara.

Introducción y Objetivos

El grupo debe aplicar las técnicas de la materia Álgebra Lineal Computacional y el módulo ALC construido durante el curso del laboratorio para la resolución de la consigna de este Trabajo Práctico.

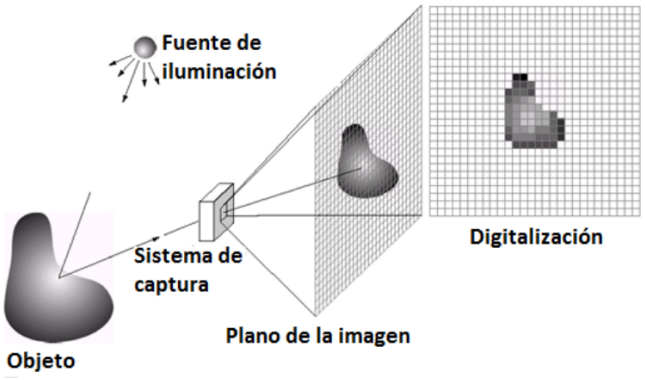


Figura 1: Proyección de un objeto sobre la retina, el plano imagen y la grilla de pixeles.

Este trabajo práctico aborda la problemática de interpretar la información espacial del mundo real capturada por una cámara, que podemos también pensarlo como una discretización 2D de un espacio 3D (ver fig. 1). En otras palabras, aquellos objetos del mundo real que son registrados por una lente, terminan perdiendo una dimensionalidad para poder representarlos en una foto o cuadro. El ejemplo más común es un cuadro de un paisaje en perspectiva. Además, esta información, que puede considerarse de carácter continuo, se termina discretizando espacialmente para obtener su representación visual sobre una grilla de pixeles... o matriz... que por suerte, en ALC, somos especialistas en matrices... :-)

Veremos a continuación la forma de recuperar la información del mundo real a partir de la simplificación que significa la digitalización de la vista. Específicamente, la idea es, por ejemplo, analizar la posición de los jugadores de la fig. 2 y responder la pregunta de si el pie del defensor habilita a los jugadores del equipo blanco.

Entre otras aplicaciones más conocidas podemos nombrar los sistemas de medición de la velocidad de automóviles en calles o rutas, o estimar la posición de jugadores en una cancha de fútbol para estimar una posición adelantada (lo que hace el VAR, bah), y veremos también una aplicación de ayuda al aterrizaje de drones.

Para la resolución de este tipo de problemas, vamos a estudiar el modelo de pinhole de la cámara, su calibración, y transformación de puntos ubicados en un plano (coplanares) mediante una homografía, o un algoritmo iterativo de ajuste de la pose de la cámara (POSIT).



Figura 2: Posible Off-Side.

1. Estimación de la pose de una cámara

1.1. Modelo de cámara *pinhole*

La Fig. 3 muestra el clásico modelo de *pinhole*. En este modelo podemos definir 2 ejes de coordenadas diferentes. El primero corresponde a las coordenadas de la cámara (X, Y, Z), que tiene su centro de proyección en el punto F, y que define un plano imagen G a una distancia focal f, que es paralelo al plano XY. Este eje coordenado tiene su eje Z, perpendicular a G, también llamado eje óptico de la cámara. Por otro lado tenemos el eje coordenado del mundo real definido por (U, V, W) y que consiste en los datos en 3 dimensiones del mundo físico real. Finalmente se define el plano imagen, definido por los ejes (u, v) que consisten en una grilla rectangular de pixeles.

Un punto M_i del espacio real, ubicado en la posición (U_i, V_i, W_i), se proyecta en el plano imagen en $m_i = (u_i, v_i)$ definido por la intersección del plano imagen G y de la recta que lo une con F. Este punto corresponde a la posición (X_i, Y_i, Z_i) en las coordenadas de la cámara, y la posición del punto m_i en estos ejes sería (X_i', Y_i', Z_i').

Definimos también un punto de referencia M_0, el origen de referencias del eje de coordenadas del objeto, con sus coordenadas en el sistema de la cámara siendo (X_0, Y_0, Z_0).

En resumen, las ecuaciones que vinculan los objetos entre los sistemas de coordenadas son las siguientes:

1. Del mundo real a las coordenadas de la cámara:

$$\begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} U_i \\ V_i \\ W_i \\ 1 \end{bmatrix}. \tag{1}$$

   con $\mathbf{R} \in \mathbb{R}^{3\times3}$ y $\mathbf{t} \in \mathbb{R}^3$ se denominan las variables extrínsecas de la cámara y definen la pose de la misma.

2. De las coordenadas de la cámara al plano imagen

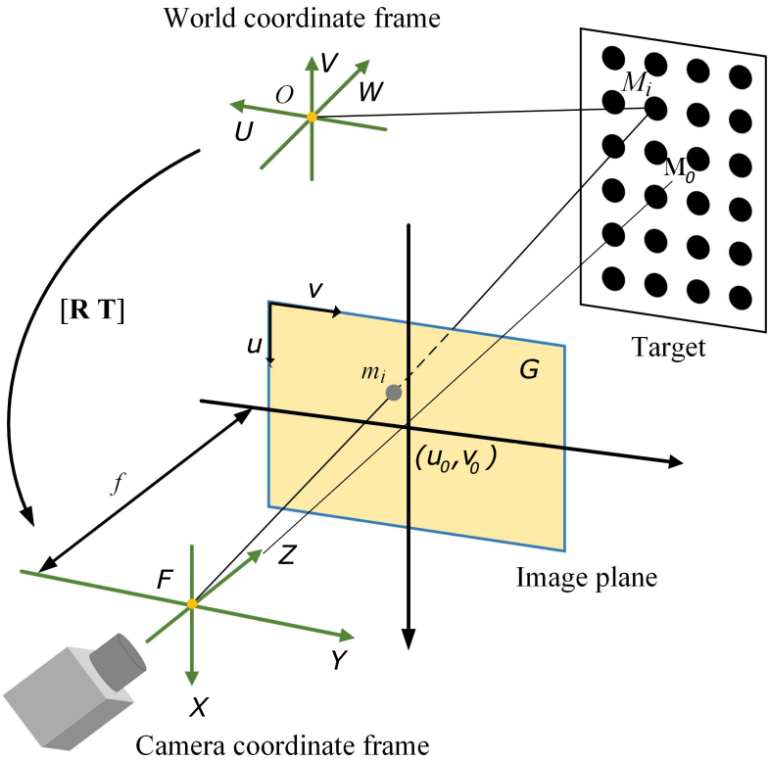


Figura 3: Modelo Pinhole.

$$\lambda \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix}. \tag{2}$$

donde $\mathbf{K}$ se define como los parámetros intrínsecos de la cámara, definiendo la discretización que sufre la transformación de puntos reales sobre la geometría de la grilla de la retina artificial.

Definición del problema de calibración: la calibración de la cámara significa calcular la matriz $\mathbf{K}$, la matriz de rotación $\mathbf{R}$ y el vector de traslación $\mathbf{t}$, a través de filmar o sacar fotografías a objetos de dimensiones conocidas. Las coordenadas (U_i, V_i, W_i) de los puntos M_i en el espacio real son conocidas. Los puntos proyectados de los M_i en el plano imagen m_i y sus coordenadas (u_i, v_i) son conocidas. Las coordenadas (X_i, Y_i, Z_i) de los puntos M_i en el sistema de coordenadas de la cámara no son conocidos porque la pose del objeto en el sistema de coordenadas de la cámara es desconocido.

Ejercicios

Parte A: Herramientas

En esta primera parte se desarrollan las herramientas de calibración de la cámara que se utilizarán a lo largo del trabajo.

Ejercicio 1 (DLT Ecuaciones formales)**.** A partir de los datos compartidos en `calib/3d` armar el sistema de la ecuación 6. Contestar las siguientes preguntas:

a) Siendo $Q \in \mathbb{R}^{2n \times 12}$, ¿qué rango debe tener para asegurar una solución no nula única a menos de un factor de escala? ¿Cuántos puntos, como mínimo, aseguran la solución y qué condición deben cumplir?

b) Encontrar el vector $\mathbf{m}$ que resuelve la minimización de la ecuación 8.

Ejercicio 2 (Calibración con puntos 3D - Implementación)**.** Utilizando los datos de la carpeta `calib/3d`:

a) Implementar la función `obtenerQ3d(imgPts, objPts)` que devuelve la matriz Q a partir de la ecuación 6.

b) A partir de lo resuelto en el Ejercicio 1 b), implementar la función `resolverDLT3D(Q)`, que a partir de Q encuentra la matriz de proyección $\mathbf{M}$.

c) Una vez recuperada $\mathbf{M}$, se la descompone en la matriz de parámetros intrínsecos $\mathbf{K}$, la matriz de rotación $\mathbf{R}$ y el vector de traslación $\mathbf{t}$, imponiendo que $\mathbf{R}$ sea ortonormal ($\mathbf{R}\mathbf{R}^T = I$). Para ello, implementar la función `devolverMatricesCalibracion3d(M)`, que devuelva $\mathbf{R}$, $\mathbf{t}$ y $\mathbf{K}$.

d) Agregar a la función `devolverMatricesCalibracion3d` un parámetro opcional que permita imponer que $\alpha_u = \alpha_v$ y/o $K_{12} = 0$.

Ejercicio 3 (Calibración con puntos coplanares)**.** Resolver la calibración para el caso de puntos coplanares, trabajando sobre los datos de la carpeta `calib/coplanar` (imágenes `Im_R_1.png` a `Im_R_20.png`). Las imágenes muestran un tablero de ajedrez plano, visto desde distintas poses, con 12×8 cuadrados, es decir 11×7 esquinas internas, y cada cuadrado de $30\,\text{mm} \times 30\,\text{mm}$ (el tablero mide entonces $360\,\text{mm} \times 240\,\text{mm}$). Tomar como sistema de coordenadas del tablero un origen en una de las esquinas internas, con ejes U y V a lo largo de los lados y $W = 0$, de modo que las esquinas internas tienen coordenadas $(30\,i,\, 30\,j)$ mm, con $i = 0, \ldots, 10$ y $j = 0, \ldots, 6$.

- ¿Qué rango debe tener $Q \in \mathbb{R}^{2n \times 9}$ para asegurar una solución única de H a menos de un factor de escala?
- Implementar la función `obtenerQ2d(imgPts, objPts)` que devuelve la matriz Q a partir de la ecuación 11.
- Implementar la función `armarA(Hlst)` que recibe una lista de matrices H_j correspondientes a las N capturas de las miras, obtenidas a partir de la resolución del sistema planteado en la ecuación 11.
- Implementar `calcularMatrizK2D(A)` que recibe la matriz A y calcula la matriz $\mathbf{K}$.
- Implementar `obtenerPose2D(Hj, K)` que devuelve la pose de la vista j como $\mathbf{R}_j$ y $\mathbf{t}_j$ a partir de la matriz $\mathbf{K}$ y la $\mathbf{H}_j$ correspondiente.

Parte B: Aplicaciones

En esta segunda parte se abordan problemas concretos de interpretación espacial a partir de imágenes.

Ejercicio 4 (Ubicación espacial en la cancha de fútbol)**.** En este ejercicio se aplicarán las herramientas de puntos coplanares para ubicar objetos sobre el campo de juego a partir de una foto. Como el campo de juego es un plano, la relación entre sus puntos y los de la imagen queda determinada por una homografía $\mathbf{H}$ (ecuación 10), que se estima a partir de correspondencias entre puntos de referencia de la cancha y sus posiciones en la foto. Las dimensiones de la cancha del Estadio Azteca, en metros, se encuentran en el archivo `estadio-azteca.png`, y se toman como sistema de coordenadas (U, V) del plano del piso. Para este ejercicio **no** es necesario conocer los parámetros intrínsecos $\mathbf{K}$ de la cámara ni descomponer $\mathbf{H}$ en $\mathbf{R}$ y $\mathbf{t}$: alcanza con $\mathbf{H}$ y su inversa.



Figura 4: Posición Jugadores.

- Para la foto `gol-enzo.png`, estimar la homografía $\mathbf{H}$ del plano de la cancha a la imagen y utilizarla para:

  a) Calcular la distancia en línea recta, medida sobre el piso, a la que se encuentra Enzo de la línea del arco cuando patea al gol. Para ello, llevar la posición de sus pies de la imagen a la cancha mediante $\mathbf{H}^{-1}$.

  b) Ubicar espacialmente todos los jugadores argentinos y dibujar un círculo de 1 metro de radio sobre el piso, como se muestra en la fig. 4. Para ello, definir el círculo en coordenadas de la cancha y proyectar sus puntos a la imagen mediante $\mathbf{H}$.

- Para la foto `posibleoffside.jpg`, estimar su propia homografía $\mathbf{H}$ y utilizarla para:

  c) Trazar una recta paralela a la línea de fondo de la cancha que toque el pie del último defensor (es decir, el penúltimo jugador adversario, contando al arquero) para determinar si los jugadores de blanco estaban en fuera de juego. La recta se define en coordenadas de la cancha y se proyecta a la imagen mediante $\mathbf{H}$.

Ejercicio 5 (Cálculo de velocidad real)**.** El objetivo del ejercicio es encontrar, frame a frame de una secuencia de imágenes, la velocidad de dos vehículos y una peatona en la calle Jerónimo Salguero, al llegar a la esquina con la calle Güemes, como se ve en la fig. 5.

Se pide en este ejercicio:



Figura 5: Vista Salguero.

a) Calcular los parámetros intrínsecos de la cámara a partir de las diferentes vistas de la esquina y las coordenadas de los puntos en el archivo `world_img_pts.csv`. Cada fila corresponde a un punto, con sus coordenadas X, Y, Z en metros y su posición (u_k, v_k) en cada una de las vistas k. Tener en cuenta que no todos los puntos tienen $Z = 0$: los puntos sobre el cordón de la vereda están a $Z = 0{,}1$ m del nivel de la calle.

b) A partir de los puntos `gs_10_09_point_correspondences_v5.txt` calcular la pose de la cámara y reproyectar los puntos sobre la imagen `img_10_18_47_351.jpg`. El archivo contiene los datos de los puntos organizados en 6 filas, con una columna por punto (51 puntos en total): la primera fila es el índice del punto (de 0 a 50), la segunda las coordenadas X, la tercera las coordenadas Y, la cuarta las coordenadas Z (en metros), la quinta las coordenadas u y la sexta las coordenadas v (en píxeles) en la imagen inicial.

c) Calcular la velocidad de los dos primeros vehículos (taxi y camioneta) y de la señora de buzo rojo que cruza la calle, a partir de las capturas de un video disponibles en esta carpeta[1]. El nombre de cada captura indica el instante en que fue registrada, que se utiliza como dato temporal para calcular la velocidad de los agentes dinámicos de la escena.

Ejercicio 6 (Aterrizaje suave)**.** En este ejercicio vamos a implementar el método de POSIT para el cálculo de la pose de la cámara con respecto a una mira de tipo H para el aterrizaje de un drone. Para este ejercicio se aporta una matriz de parámetros intrínsecos de la cámara apostada en el vehículo (archivo `K.csv`), y la lista de puntos reales con sus coordenadas en metros. Como vimos en la sección 4, POSIT aporta 2 soluciones para un problema coplanar. En esta implementación, vamos a simplificar el algoritmo siempre seleccionando en cada iteración la solución que minimice el error cuadrático medio. También puede ser que en una solución no todos los puntos queden frente a la cámara, con lo cual su valor de z podría ser negativo. En este caso, esa solución no tiene que tomarse en cuenta.

Se pide en el ejercicio:

[1] `https://drive.google.com/drive/folders/1hOW43Jy1gA6d_Xg-7bfuZu9eAxv4RViP?usp=sharing`

a) Elegir convenientemente los puntos a etiquetar en la imagen capturada por la cámara del drone (`row_00248.png`) para calcular la pose.

b) Implementar la función `calculoLambdaMu(I0,J0)` que devuelve λ y μ de la ecuación 16.

c) Implementar la función `calcularSalidaPOSIT(Is, Js, fx, m0, M0)` que devuelve una pose correspondiente a una posible solución $\mathbf{i}$, $\mathbf{j}$, $\mathbf{k}$, Z_0, $\mathbf{R}$, $\mathbf{t}$ de (I_s, J_s).

d) Implementar la función `evaluarPose(R, t, obj_vecs, img_vecs, K)` que devuelve la cantidad de proyecciones válidas de `obj_vecs`, y el RMS obtenido con respecto a las posiciones originales en `img_vecs`.

e) Implementar la función `aplicarPOSIT(obj_vecs, img_vecs, K, MaxIt, atol)` que devuelva la pose de la cámara ($\mathbf{R}$ y $\mathbf{t}$) luego de realizar `MaxIt` iteraciones o salir por convergencia, es decir, cuando no hay mejora en el RMS de una iteración a otra (se considera que no hay mejora cuando la diferencia del RMS entre dos iteraciones es menor a `atol`).

f) Estimar la matriz de rotación y traslación para ubicar al drone exactamente sobre la letra H y que la cámara la vea vertical (el punto blanco marca la parte superior izquierda).

Template-alumnos, Defensa Oral y Entrega

En el campus, en los recursos para el TP van a encontrar los siguientes archivos:

- Archivo `tp2c2026.pdf`: este archivo de enunciado del TP.
- Archivo `template-alumnos.zip`: archivo de recursos con el siguiente contenido:
  - Folder calib
  - Folder futbol
  - Folder salguero
  - Folder heli
  - Notebook: `grupo_XX_TP.ipynb`

La entrega se realizará a través del campus virtual de la materia además de una defensa oral grupal del trabajo. Las fechas y formatos son los siguientes:

- Fecha de entrega: hasta el Viernes **6 de Noviembre** a las 23:59 hs.
- Fecha de defensa oral: Martes **17 de Noviembre**.
- Formato item `.py`: archivo biblioteca con todas las funciones pedidas en el TP y del labo. Nombre del archivo: alc.py.
- Formato items `.ipynb`: Jupyter Notebook que servirá como entregable de las salidas esperadas de los ejercicios de este enunciado. Los subtimes de la notebook no siguen necesariamente a los ejercicios, y son una guía. Los que valen son aquellos presentados en este manuscrito.

Prestar especial atención a las siguientes indicaciones:

- El TP se realizará en grupos de cuatro personas. Indicar a los docentes si el grupo declarado en el 'Foro de Grupos de TP' no ha sido aún creado. **Importante:** es indispensable realizar la inscripción previa del grupo para poder hacer el envío a través del campus. Los grupos o personas no inscriptas en grupos no estarán habilitadas en el formulario de carga del TP. No se aceptarán envíos por email.
- Leer el enunciado completo antes de comenzar a generar código y sacarse todas las dudas de cada ítem antes de implementar. Para obtener un código más legible y organizado, pensar de antemano qué funciones deberán implementarse y cuáles podrían reutilizarse.
- El código debe estar correctamente comentado. Cada función definida debe contener un encabezado donde se explique los parámetros que recibe y qué se espera que retorne. Además las secciones de código dentro de la función deben estar debidamente comentados. Los nombre de las variables deben ser explicativos.
- **Uso de librerías.** Del código entregado solo se permite usar las siguientes funciones de librerías externas, y nada más:
  - **Arrays (NumPy):** `np.array`, `np.asarray`, `np.zeros`, `np.ones`, `np.eye`, `np.arange`, `np.hstack`, `np.vstack`, `reshape`, `.T`, `np.float32`, `np.int32`, `np.set_printoptions`, y la indexación y el slicing de arrays.

- **Operaciones elemento a elemento (NumPy):** `+`, `-`, `*`, `/`, `**`, `np.sqrt`, `np.abs`, `np.sum`, `np.mean`, `np.max`, `np.min`, `np.all`, `np.any` y `np.diff`.
- **Lectura de archivos:** `np.loadtxt`, `pd.read_csv`, `os.listdir` y `os.path.join`.
- **Lectura de imágenes:** `cv2.imread` y `matplotlib.image.imread`.
- **Gráficos (Matplotlib):** `plt.figure`, `plt.subplots`, `plt.imshow`, `plt.plot`, `plt.scatter`, `plt.text`, `plt.Circle`, `plt.title`, `plt.legend`, `plt.tight_layout`, `plt.savefig` y `plt.show`.

Todo lo demás debe ser implementado por el grupo, reutilizando el módulo ALC construido durante el laboratorio. En particular, no se permite el uso de funciones de librerías para el producto matricial (`@`, `np.dot`, `np.matmul` y similares), las factorizaciones QR, SVD y Cholesky, la inversa, el determinante, la pseudoinversa, las normas, el número de condición, el rango, los autovalores, el producto vectorial, ni las funciones de OpenCV de calibración, pose y proyección (`calibrateCamera`, `solvePnP`, `projectPoints`, `findHomography`, `Rodrigues`, etc.). Si bien estas funciones no están permitidas en el código entregado, sí pueden usarse durante el desarrollo para verificar los resultados obtenidos con las implementaciones propias.

- Las conclusiones y razonamientos que respondan los ejercicios, o cualquier experimentación agregada, debe estar debidamente explicada en bloques de texto de las notebooks (markdown cells), separado de los bloques de código. Aprovechen a utilizar código LaTeX si necesitan incluir fórmulas, sobre todo para la demostración del primer ejercicio.

Régimen de aprobación del TP

- La aprobación del Trabajo Práctico es Obligatoria para la aprobación de la cursada de la materia.
- Todos los grupos deben entrega el TP en la 1ra fecha. No entregar significa el desaprobado del TP.
- Para poder acceder a la defensa Oral y a una re-entrega, los grupos tienen que tener correctos al menos 3 ejercicios. Si no tienen correctos al menos 3 ejercicios el TP está directamente desaprobado.
- Durante la Defensa Oral se va a evaluar el conocimiento del grupo sobre la resolución del TP. No aprobar el Oral puede incidir sobre la posibilidad de Promoción de TODOS los integrantes del grupo.
- Se busca que todos los ejercicios del TP entregados por el grupo estén correctos. Para ello v a haber una instancia de re-entrega. No hacer la re-entrega con las correcciones pedidas por el docente evaluador significa la desaprobación del TP.

Marco teórico

2. Calibración de la cámara

La calibración la vamos a resolver mediante Transformación Lineal Directa (DLT) usando

- Usando objetos 3D (método de Tsai),
- Utilizando grillas planas (método de Zhang).

2.1. DLT en 3 dimensiones

Este método fue propuesto en 1987 por Tsai[2] y consiste en utilizar puntos de control en una mira de calibración 3D y sus coordenadas 2D en la imagen para obtener la calibración.

La idea del DLT es acomodar las ecuaciones de proyección perspectiva como una ecuación homogénea y resolver con métodos estándar. Para una correspondencia genérica de los puntos 3D–2D:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} U \\ V \\ W \\ 1 \end{bmatrix}. \tag{3}$$

Con

$$\mathbf{K} = \begin{bmatrix} \alpha_u & \gamma & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix}, \tag{4}$$

donde $\gamma = K_{12}$ es el sesgo (*skew*) entre los ejes de la imagen. En la práctica suele ser $\gamma \approx 0$ (y a veces también $\alpha_u = \alpha_v$), lo que puede imponerse como restricción adicional.

Definimos la matriz de proyección como

$$\mathbf{M} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} = \begin{bmatrix} \boldsymbol{m}_1^\mathsf{T} \\ \boldsymbol{m}_2^\mathsf{T} \\ \boldsymbol{m}_3^\mathsf{T} \end{bmatrix}.$$

siendo $\boldsymbol{m}_j^\mathsf{T}$ la fila j-ésima de $\mathbf{M}$. Y finalmente, para un punto $P = \begin{bmatrix} U & V & W & 1 \end{bmatrix}^\mathsf{T}$, tenemos

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{m}_1^\mathsf{T} \\ \boldsymbol{m}_2^\mathsf{T} \\ \boldsymbol{m}_3^\mathsf{T} \end{bmatrix} P$$

Despejando las coordenadas de un pixel en función de las coordenadas homogéneas nos da:

$$u = \frac{\lambda u}{\lambda} = \frac{\boldsymbol{m}_1^\mathsf{T} P}{\boldsymbol{m}_3^\mathsf{T} P}, \qquad v = \frac{\lambda v}{\lambda} = \frac{\boldsymbol{m}_2^\mathsf{T} P}{\boldsymbol{m}_3^\mathsf{T} P}.$$

$$(\boldsymbol{m}_1^\mathsf{T} - u_i \boldsymbol{m}_3^\mathsf{T}) P_i = 0, \qquad (\boldsymbol{m}_2^\mathsf{T} - v_i \boldsymbol{m}_3^\mathsf{T}) P_i = 0.$$

[2] Roger Y. Tsai, "A Versatile Camera Calibration Technique for High-Accuracy 3D Machine Vision Metrology Using Off-the-Shelf TV Cameras and Lenses," *IEEE Journal of Robotics and Automation*, 1987

Vamos a reacomodar las filas de la matriz $\mathbf{M}$ como columnas, con lo cual definimos un vector:

$$\boldsymbol{m} = \begin{bmatrix} \boldsymbol{m}_1 \\ \boldsymbol{m}_2 \\ \boldsymbol{m}_3 \end{bmatrix}$$

Matricialmente:

$$\begin{bmatrix} P_i^\mathsf{T} & \mathbf{0}^\mathsf{T} & -u_i P_i^\mathsf{T} \\ \mathbf{0}^\mathsf{T} & P_i^\mathsf{T} & -v_i P_i^\mathsf{T} \end{bmatrix} \boldsymbol{m} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \tag{5}$$

Y para n puntos, todas las ecuaciones pueden concatenarse como

$$\begin{bmatrix} P_1^\mathsf{T} & \mathbf{0}^\mathsf{T} & -u_1 P_1^\mathsf{T} \\ \mathbf{0}^\mathsf{T} & P_1^\mathsf{T} & -v_1 P_1^\mathsf{T} \\ \vdots & \vdots & \vdots \\ P_n^\mathsf{T} & \mathbf{0}^\mathsf{T} & -u_n P_n^\mathsf{T} \\ \mathbf{0}^\mathsf{T} & P_n^\mathsf{T} & -v_n P_n^\mathsf{T} \end{bmatrix} \boldsymbol{m} = \mathbf{0}. \tag{6}$$

Explícitamente:

$$\begin{bmatrix} U_1 & V_1 & W_1 & 1 & 0 & 0 & 0 & 0 & -u_1U_1 & -u_1V_1 & -u_1W_1 & -u_1 \\ 0 & 0 & 0 & 0 & U_1 & V_1 & W_1 & 1 & -v_1U_1 & -v_1V_1 & -v_1W_1 & -v_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ U_n & V_n & W_n & 1 & 0 & 0 & 0 & 0 & -u_nU_n & -u_nV_n & -u_nW_n & -u_n \\ 0 & 0 & 0 & 0 & U_n & V_n & W_n & 1 & -v_nU_n & -v_nV_n & -v_nW_n & -v_n \end{bmatrix} \boldsymbol{m} = \mathbf{0}$$

DLT plantea entonces el sistema homogéneo

$$Q\mathbf{m} = \mathbf{0},$$

donde Q es conocido, que se obtiene de los datos de los puntos de referencia, y $\boldsymbol{m}$ es la matriz de proyección vectorizada a averiguar. Con datos ideales y suficientes, existiría un $\mathbf{m} \neq 0$ exactamente en el núcleo de Q.

Pero con datos reales, ruido, errores de localización de puntos, etc., normalmente no existe un vector no nulo que satisfaga exactamente $Q\mathbf{m} = \mathbf{0}$, lo que nos lleva a reemplazar la igualdad exacta a plantear la minimización:

$$\min_{\mathbf{m}} \|Q\boldsymbol{m}\|^2, \text{ sujeto a } \|\boldsymbol{m}\| = 1 \tag{7}$$

se puede deducir que el sistema anterior queda formalmente planteado como:

$$\min_{\|\boldsymbol{m}\|=1} \mathbf{m}^T Q^T Q \mathbf{m} \tag{8}$$

Lo que DLT hace aquí puede llamarse mínimos cuadrados homogéneos con restricción de norma, que en lugar de resolver la forma convencional $\min_x \|Qx - b\|^2$, resuelve el sistema de la ecuación 7.

Una vez que se determina $\boldsymbol{m}$, los parámetros intrínsecos y extrínsecos se pueden recobrar de las ecuaciones 4.

$$\begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} = \begin{bmatrix} \alpha_u r_{11} + \gamma r_{21} + u_0 r_{31} & \alpha_u r_{12} + \gamma r_{22} + u_0 r_{32} & \alpha_u r_{13} + \gamma r_{23} + u_0 r_{33} & \alpha_u t_1 + \gamma t_2 + u_0 t_3 \\ \alpha_v r_{21} + v_0 r_{31} & \alpha_v r_{22} + v_0 r_{32} & \alpha_v r_{23} + v_0 r_{33} & \alpha_v t_2 + v_0 t_3 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix}$$

2.2. Calibración desde puntos coplanares (Zhang's Algorithm)

El método de Tsai requiere una mira de puntos 3D, lo que no es siempre práctico. Toolboxes modernas de calibración como la de Matlab o OpenCV, usan múltiples vistas de una grilla coplanar, como un tablero de ajedrez, y se basan en el artículo del 2000[3].

Para aplicar el algoritmo DLT para una grilla coplanar comenzamos desde la ecuación de proyección perspectiva y planteando los puntos coplanares $W_i = 0.$, lo que nos da:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} U \\ V \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_u & \gamma & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & t_1 \\ r_{21} & r_{22} & t_2 \\ r_{31} & r_{32} & t_3 \end{bmatrix} \begin{bmatrix} U \\ V \\ 1 \end{bmatrix}. \tag{9}$$

La expresión anterior puede escribirse como una homografía:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} U \\ V \\ 1 \end{bmatrix},$$

donde

$$\mathbf{H} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} = \mathbf{K} \begin{bmatrix} \boldsymbol{r}_1 & \boldsymbol{r}_2 & \boldsymbol{t} \end{bmatrix} = \begin{bmatrix} \boldsymbol{h}_1^\mathsf{T} \\ \boldsymbol{h}_2^\mathsf{T} \\ \boldsymbol{h}_3^\mathsf{T} \end{bmatrix}. \tag{10}$$

con $\boldsymbol{h}_j^\mathsf{T}$ la fila j-ésima de $\mathbf{H}$

Para el punto $P_i = \begin{bmatrix} U_i & V_i & 1 \end{bmatrix}^\mathsf{T}$ que mapea un punto $m_i = \begin{bmatrix} u_i & v_i & 1 \end{bmatrix}^\mathsf{T}$, utilizando 10 y despejando en 9, nos queda:

$$u_i = \frac{\boldsymbol{h}_1^\mathsf{T} P_i}{\boldsymbol{h}_3^\mathsf{T} P_i}, \qquad v_i = \frac{\boldsymbol{h}_2^\mathsf{T} P_i}{\boldsymbol{h}_3^\mathsf{T} P_i}.$$

Por lo tanto,

$$(\boldsymbol{h}_1^\mathsf{T} - u_i \boldsymbol{h}_3^\mathsf{T}) P_i = 0, \qquad (\boldsymbol{h}_2^\mathsf{T} - v_i \boldsymbol{h}_3^\mathsf{T}) P_i = 0.$$

Reacomodando las ecuaciones para el punto i,

$$\begin{bmatrix} P_i^\mathsf{T} & \mathbf{0}^\mathsf{T} & -u_i P_i^\mathsf{T} \\ \mathbf{0}^\mathsf{T} & P_i^\mathsf{T} & -v_i P_i^\mathsf{T} \end{bmatrix} \begin{bmatrix} \boldsymbol{h}_1 \\ \boldsymbol{h}_2 \\ \boldsymbol{h}_3 \end{bmatrix} = \begin{bmatrix} 0 \\ .. \\ 0 \end{bmatrix}$$

Y apilando para los N puntos,

$$\begin{bmatrix} P_1^\mathsf{T} & \mathbf{0}^\mathsf{T} & -u_1 P_1^\mathsf{T} \\ \mathbf{0}^\mathsf{T} & P_1^\mathsf{T} & -v_1 P_1^\mathsf{T} \\ & & \\ P_N^\mathsf{T} & \mathbf{0}^\mathsf{T} & -u_N P_N^\mathsf{T} \\ \mathbf{0}^\mathsf{T} & P_N^\mathsf{T} & -v_N P_N^\mathsf{T} \end{bmatrix} \begin{bmatrix} \boldsymbol{h}_1 \\ \boldsymbol{h}_2 \\ \boldsymbol{h}_3 \end{bmatrix} = \begin{bmatrix} 0 \\ .. \\ 0 \end{bmatrix} \implies Q\boldsymbol{h} = \mathbf{0} \tag{11}$$

donde Q es conocido y $\boldsymbol{h}$ es la homografía $\mathbf{H}$ vectorizada que tenemos que encontrar y que ya hemos visto en la sección precedente.

[3] Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2000.

La metodología convencional utilizada para la calibración de la cámara consiste en tomar varias fotografías de una plantilla plana desde diferentes ángulos y haciéndola girar siempre con todas las esquinas a la vista. Cada vista j tiene diferentes homografías $\mathbf{H}_j$, y por lo tanto diferentes $\mathbf{R}_j, \mathbf{t}_j$, pero el mismo $\mathbf{K}$.

Veamos cómo sería el algoritmo completo:

a) Estimar la homografía $\mathbf{H}_j$ para cada vista j-ésima usando el algoritmo DLT.

b) Para cada homografía tenemos $\mathbf{H}_j \sim \mathbf{K}\begin{bmatrix}\boldsymbol{r}_1 & \boldsymbol{r}_2 & \boldsymbol{t}\end{bmatrix}$. Si escribimos las columnas de $\mathbf{H}$ como $\mathbf{H} = \begin{bmatrix}\boldsymbol{c}_1 & \boldsymbol{c}_2 & \boldsymbol{c}_3\end{bmatrix}$, salvo un factor de escala s

$$\boldsymbol{c}_1 = s\mathbf{K}\boldsymbol{r}_1, \qquad \boldsymbol{c}_2 = s\mathbf{K}\boldsymbol{r}_2, \tag{12}$$

c) Definimos una matriz simétrica $B = \mathbf{K}^{-\top}\mathbf{K}^{-1}$ que bajo las restricciones de ortogonalidad e igualdad de normas

$$\boldsymbol{r}_1^\top \boldsymbol{r}_1 = \boldsymbol{r}_2^\top \boldsymbol{r}_2 = 1, \qquad \boldsymbol{r}_1^\top \boldsymbol{r}_2 = 0$$

se convierten en

$$\boldsymbol{c}_1^\top B \boldsymbol{c}_1 - \boldsymbol{c}_2^\top B \boldsymbol{c}_2 = 0, \qquad \boldsymbol{c}_1^\top B \boldsymbol{c}_2 = 0. \tag{13}$$

cada homografía aporta exactamente dos ecuaciones lineales en las seis componentes independientes de B. De esta manera nos hemos despojado de las variables $\mathbf{r}_1$ y $\mathbf{r}_2$ y las incógnitas que nos quedan resolver dependen de $\mathbf{K}$ y de los valores de las columnas de $\mathbf{H}$.

d) Una vez obtenidas todas las $\mathbf{H}_j$, vamos a encontrar la matriz $\mathbf{K}$, planteando un sistema $A\mathbf{b} = 0$, donde el vector $\mathbf{b}$ (incógnita) se obtiene a partir de la B simétrica

$$B = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix}$$

que reacomodando los valores de acuerdo a las ecuaciones precedentes nos queda

$$\boldsymbol{b} = \begin{bmatrix} B_{11} \\ B_{12} \\ B_{22} \\ B_{13} \\ B_{23} \\ B_{33} \end{bmatrix}$$

e) Queda ahora construir la matriz A. Para dos columnas $\mathbf{c}_i$, $\mathbf{c}_j$ de $\mathbf{H}$ se define un v_{ij} (ver [4] sección 3) que permite escribir $\boldsymbol{c}_i^\top B \boldsymbol{c}_j = \boldsymbol{v}_{ij}^\top \boldsymbol{b}$. Las dos restricciones 13 de una vista quedan definidas como

$$\boldsymbol{v}_{12}^\top \boldsymbol{b} = 0 \qquad (\boldsymbol{v}_{11} - \boldsymbol{v}_{22})^\top \boldsymbol{b} = 0,$$

y que corresponden a dos filas de la matriz A. Luego, las otras filas de A se arman con las homografías de todas las vistas.

f) Apilar $2N$ ecuaciones de N vistas para obtener A de dimensión $\mathbb{R}^{2N\times 6}$. Luego, resolver $\mathbf{b}$ de $A\boldsymbol{b} = \mathbf{0}$ usando mínimos cuadrados homogéneos con restricción de norma. A partir del $\mathbf{b}$ reconstruir B. Como $\mathbf{b}$ se determina solo salvo un factor de escala (y su signo es arbitrario), se elige el signo que haga a B definida positiva. Como $\mathbf{K}$ tiene 5 parámetros (4 si se impone $\gamma = 0$) y cada vista aporta 2 ecuaciones, se necesitan al menos $N \geq 3$ vistas ($N \geq 2$ si $\gamma = 0$), con poses no degeneradas.

g) A partir de B se usa la factorización de Cholesky $B = \mathbf{L}\mathbf{L}^\mathsf{T}$, con $\mathbf{L}$ triangular inferior. Como $B = \mathbf{K}^{-\mathsf{T}}\mathbf{K}^{-1}$ y $\mathbf{K}^{-1}$ es triangular superior, resulta $\mathbf{L} = \mathbf{K}^{-\mathsf{T}}$, es decir, $\mathbf{K} = (\mathbf{L}^\mathsf{T})^{-1}$. Como B se conoce solo salvo escala, se normaliza $\mathbf{K}$ dividiéndola por K_{33} para que $K_{33} = 1$.

h) Los parámetros extrínsecos de la vista j se pueden obtener usando $\mathbf{K}$ y la matriz $\mathbf{H}_j = \begin{bmatrix}\boldsymbol{c}_1 & \boldsymbol{c}_2 & \boldsymbol{c}_3\end{bmatrix}$:

$$\lambda = \frac{1}{\|\mathbf{K}^{-1}\mathbf{c}_1\|} = \frac{1}{s} \qquad \boldsymbol{r}_1 = \lambda\mathbf{K}^{-1}\mathbf{c}_1, \qquad \boldsymbol{r}_2 = \lambda\mathbf{K}^{-1}\mathbf{c}_2, \qquad \boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2.$$

Finalmente,

$$\mathbf{R}_j = \begin{bmatrix}\boldsymbol{r}_1 & \boldsymbol{r}_2 & \boldsymbol{r}_3\end{bmatrix}, \qquad \mathbf{t}_j = \lambda\mathbf{K}^{-1}\mathbf{c}_3,$$

y se aplican restricciones a la matriz de rotación.

2.3. Normalización isotrópica

En [1], Hartley demuestra que normalizar las coordenadas homogéneas mejora el número de condición de $Q^\mathsf{T}Q$ estabilizando los resultados del cálculo de la pose[4].

De la ecuación 10, buscamos la correspondencia entre los puntos del plano P_i y los puntos de la imagen m_i.

Si aplicamos funciones de normalización $\mathcal{T}_{img}$ y $\mathcal{T}_{world}$ tales que:

$$\check{m}_i = \mathcal{T}_{img}m_i, \qquad \check{P}_i = \mathcal{T}_{world}P_i$$

con DLT estimamos un

$$\check{m}_i \sim \check{\mathbf{H}}\check{P}_i,$$

con lo cual, la homografía en coordenadas originales sería

$$\mathbf{H} = \mathcal{T}_{img}^{-1}\check{\mathbf{H}}\mathcal{T}_{world} \tag{14}$$

El escalamiento de Hartley está apoyado en 3 puntos:

1. Hay una traslación de los puntos de manera que su centroide cae en el origen de coordenadas,
2. Todos los puntos son normalizados de manera que el promedio de sus distancias al origen es $\sqrt{2}$.
3. Se calculan las dos transformaciones $\mathcal{T}_{img}$ y $\mathcal{T}_{world}$ para los dos sistemas.

[4]En el caso de Hartley, su problemática consiste en hacer corresponder las coordenadas de dos imágenes, pero el análisis es válido para nosotros también

3. Disponibilidad limitada de puntos

Introducción

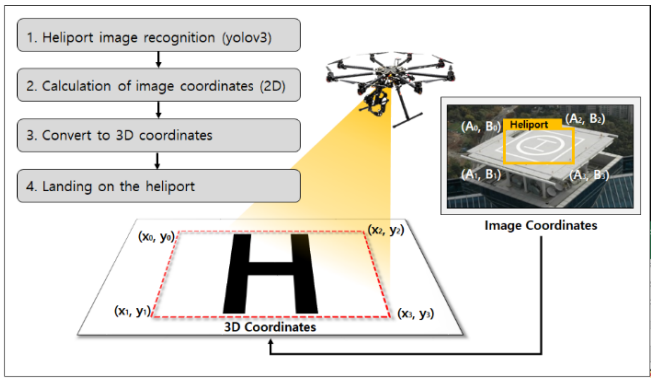


Figura 6: Imagen de reconocimiento de marca de suelo para aterrizaje automático (figura de [2]).

Llegamos a esta sección luego de analizar dos metodologías para calibrar la cámara y estimar su pose, que tienen como limitación la necesidad de un número mínimo de puntos asociados. En el caso de que la cantidad de puntos sea limitada, por ejemplo en la Fig. 6, donde la marca del suelo tendría solo 4 puntos coplanares, el drone o helicóptero debe resolver su pose a partir de esta cantidad de puntos, quizás además ayudado por una IMU (Inertial Measurement Unit).

Método POSIT

El algoritmo POSIT (*Pose from Orthography and Scaling with Iterations*) es capaz de estimar la pose de una cámara, es decir, su orientación y posición relativas respecto de un objeto conocido, a partir de correspondencias entre 4 o más puntos 3D del objeto ubicados sobre un mismo plano y sus proyecciones 2D en la imagen.

POSIT parte de una aproximación inicial conocida como proyección ortográfica escalada. Se asume que todos los puntos del objeto tienen profundidades cercanas a la profundidad del punto de referencia:

$$Z_i \approx Z_0.$$

Se supone conocido el modelo de cámara pinhole, en particular la distancia focal f y el centro de imagen $C = (c_u, c_v)$. Dado un conjunto de puntos del objeto

$$M_0, M_1, \ldots, M_n,$$

se toma M_0 como punto de referencia, con las coordenadas $[X_0, Y_0, Z_0]$, y se expresa cada punto en el sistema del objeto mediante

$$\boldsymbol{a}_i = \overrightarrow{M_0 M_i} = \begin{bmatrix} U_i \\ V_i \\ W_i \end{bmatrix}.$$

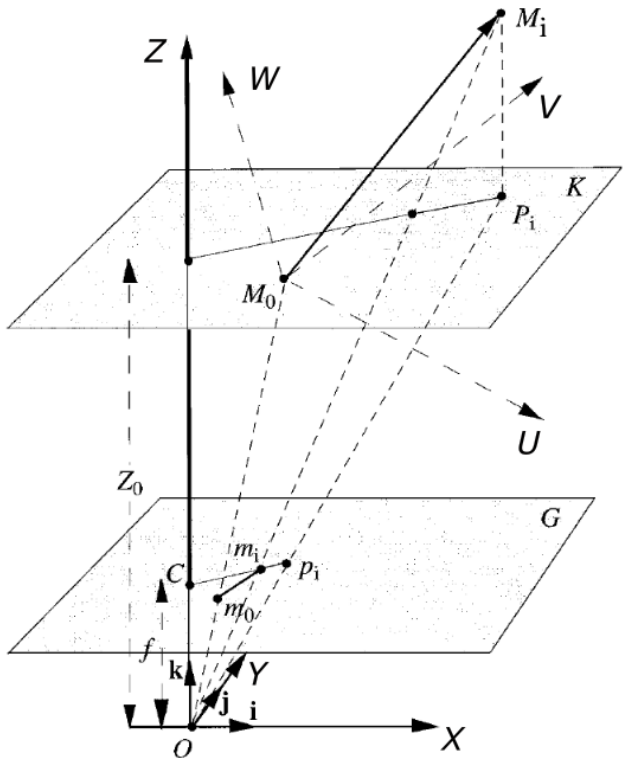


Figura 7: Geometría del método POSIT. Sistema de la cámara $OXYZ$, con ejes **i**, **j**, **k**, y plano imagen G a distancia focal f y centro C. Sistema del objeto con origen en el punto de referencia M_0 y ejes U, V, W. El plano K pasa por M_0 y es paralelo a G. P_i es la proyección de M_i sobre K; m_i es la imagen en perspectiva de M_i y p_i la de P_i (proyección ortográfica escalada).

Las coordenadas observadas en la imagen, medidas respecto del centro principal, se denotan por

$$m_i = (x_i, y_i).$$

De la ecuación 1, la relación entre las coordenadas de un punto del objeto M_i en el sistema de coordenadas de la cámara y objeto se expresa como:

$$\begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} = \begin{bmatrix} \mathbf{i}^T \\ \mathbf{j}^T \\ \mathbf{k}^T \end{bmatrix} \begin{bmatrix} U_i \\ V_i \\ W_i \end{bmatrix} + \begin{bmatrix} X_0 \\ Y_0 \\ Z_0 \end{bmatrix} \tag{15}$$

donde $\boldsymbol{i}$, $\boldsymbol{j}$ y $\boldsymbol{k}$ son los ejes de la cámara expresados en el sistema del objeto, y T, el vector de traslación, se define como el vector OM_0, el vector desde el centro de coordenadas hasta el punto de referencia M_0 (posición del punto de referencia M_0 en coordenadas de cámara). Como la imagen del punto M_0 es el punto conocido m_0, el vector de traslación T está alineado con Om_0 y es igual a $(Z_0/f)Om_0$.

Aproximación de proyección ortográfica escalada

El problema es no lineal porque las coordenadas proyectadas dependen de cocientes que involucran la profundidad Z_i. Finalmente, para calcular la traslación del objeto, precisamos obtener su coordenada Z_0, quedando la pose definida completamente además a partir de i, j, x_0, y_0.

Se define la pose exacta como la pose del objeto por el cual los puntos M_i están en la línea de vista del punto m_i sobre el plano de la imagen. Esta condición puede ser expresada por las igualdades siguientes:

$$x_0 = f\frac{X_0}{Z_0}, \qquad x_i = f\frac{X_i}{Z_i}.$$

y de manera similar para la coordenada y. Si expandimos la segunda igualdad a partir de la ecuación 15 en:

$$x_i = f\frac{M_0M_i \cdot \mathbf{i} + X_0}{M_0M_i \cdot \mathbf{k} + Z_0} = (M_0M_i \cdot \frac{f}{Z_0}\mathbf{i} + x_0)/(\frac{1}{Z_0}M_0M_i \cdot \mathbf{k} + 1)$$

Entonces, la condición necesaria y suficiente para una pose definida por i, j, x_0, y_0 y Z_0, sea la pose exacta si las siguientes ecuaciones se definen para todos los puntos M_i:

$$M_0M_i \cdot \mathbf{I} = x_i(1 + \epsilon_i) - x_0,$$

$$M_0M_i \cdot \mathbf{J} = y_i(1 + \epsilon_i) - y_0,$$

con las siguiente igualdades

$$\boldsymbol{I} = \frac{f}{Z_0}\boldsymbol{i}, \qquad \boldsymbol{J} = \frac{f}{Z_0}\boldsymbol{j},$$

los factores de corrección de perspectiva

$$\varepsilon_i = \frac{1}{Z_0}\boldsymbol{a}_i^T\boldsymbol{k}.$$

y $\mathbf{k} = \mathbf{i} \times \boldsymbol{j}$.

Como

$$Z_i = Z_0(1 + \varepsilon_i),$$

las ecuaciones de proyección pueden reescribirse como

$$\boldsymbol{a}_i^T\boldsymbol{I} = x_i(1 + \varepsilon_i) - x_0,$$

$$\boldsymbol{a}_i^T\boldsymbol{J} = y_i(1 + \varepsilon_i) - y_0.$$

Si los valores de ε_i fueran conocidos, estas ecuaciones serían lineales en las incógnitas $\boldsymbol{I}$ y $\boldsymbol{J}$.

El paso POS: resolución lineal

En cada iteración, POSIT fija los valores actuales de ε_i y resuelve dos sistemas lineales. Definimos la matriz

$$A = \begin{bmatrix} \boldsymbol{a}_1^T \\ \boldsymbol{a}_2^T \\ \vdots \\ \boldsymbol{a}_n^T \end{bmatrix},$$

También definimos los términos del lado derecho:

$$b_i^{(u)} = x_i(1+\varepsilon_i) - x_0, \qquad b_i^{(v)} = y_i(1+\varepsilon_i) - y_0.$$

Apilando estos términos para todos los puntos:

$$\boldsymbol{b}^{(u)} = \begin{bmatrix} b_1^{(u)} \\ b_2^{(u)} \\ \vdots \\ b_n^{(u)} \end{bmatrix}, \qquad \boldsymbol{b}^{(v)} = \begin{bmatrix} b_1^{(v)} \\ b_2^{(v)} \\ \vdots \\ b_n^{(v)} \end{bmatrix}.$$

Entonces el paso POS consiste en resolver

$$A\boldsymbol{I} = \boldsymbol{b}^{(u)}, \qquad A\boldsymbol{J} = \boldsymbol{b}^{(v)}.$$

En general, estos sistemas se resuelven por mínimos cuadrados usando la pseudo-inversa:

$$\boldsymbol{I}_0 = A^+\boldsymbol{b}^{(u)}, \qquad \boldsymbol{J}_0 = A^+\boldsymbol{b}^{(v)}.$$

Este paso se denomina POS, *Pose from Orthography and Scaling*. A^+ es la pseudo-inversa de A, calculable mediante SVD.

4. Caso coplanar

Cuando los puntos del objeto son coplanares, la matriz A tiene rango 2 y el sistema está mal condicionado no pudiendo determinar de forma única los vectores $\boldsymbol{I}$ y $\boldsymbol{J}$. Entonces se precisan definir restricciones adicionales.

Sea $\boldsymbol{n}$ un vector unitario normal al plano del objeto (su sentido puede tomarse en cualquiera de las dos direcciones), es decir, un vector del núcleo de A:

$$A\boldsymbol{n} = \boldsymbol{0}.$$

Entonces las soluciones posibles pueden escribirse como

$$\boldsymbol{I} = \boldsymbol{I}_0 + \lambda\boldsymbol{n},$$

$$\boldsymbol{J} = \boldsymbol{J}_0 + \mu\boldsymbol{n}.$$

Para que $\boldsymbol{I}$ y $\boldsymbol{J}$ correspondan a los dos primeros ejes de una rotación, deben satisfacer:

$$\boldsymbol{I}^T\boldsymbol{J} = 0,$$

$$\|\boldsymbol{I}\|^2 = \|\boldsymbol{J}\|^2.$$

Estas dos restricciones permiten calcular los escalares λ y μ. En general aparecen dos soluciones, que corresponden a dos poses simétricas compatibles con una aproximación ortográfica escalada. Para encontrar estos valores fijamos las restricciones: (1) $\mathbf{I}$ y $\mathbf{J}$ son perpendiculares, y (2) deben tener el mismo módulo. El punto (1) lleva a: $\lambda\mu = -\mathbf{I}_0 \cdot \mathbf{J}_0$, y el punto (2) se cumple si: $\lambda^2 - \mu^2 = \|\mathbf{J}_0\|^2 - \|\mathbf{I}_0\|^2$.

En [3] proponen un método basado en el cuadrado del número complejo $C = \lambda + i\mu$:

$$C^2 = \lambda^2 - \mu^2 + 2i\ \lambda\mu \tag{16}$$

$$\implies C^2 = \|\mathbf{J}_0\|^2 - \|\mathbf{I}_0\|^2 - 2i\ \mathbf{I}_0 \cdot \mathbf{J}_0 \tag{17}$$

y de aquí se encuentran λ y μ que son la parte real e imaginaria de las raíces cuadradas de un número complejo C^2.

De las ecuaciones 16 tenemos dos soluciones, o sea dos poses. En este caso, el algoritmo precisa explorar ambas opciones que terminan en un árbol de poses, definiendo 2^n poses luego de n iteraciones. En la práctica, vamos a seguir una o dos ramas dependiendo de una de dos situaciones.

- En el primer caso, de las 2 poses encontradas, una es descartada ya que su utilización ubica a puntos de la escena detrás de la cámara. Esto termina ocurriendo a cada iteración y permite recorrer una única rama del árbol.
- En el segundo caso: ambas poses de la primera iteración son válidas. En el segundo paso, solo se perseguirán las ramas pertenecientes a las mejores poses. La mejor pose se decide como el promedio de las distancias entre los puntos de las imágenes y sus proyecciones a partir de aplicar la pose.

Recuperación de la pose aproximada

Una vez obtenidos $\boldsymbol{I}$ y $\boldsymbol{J}$, se recupera la profundidad del punto de referencia mediante

$$Z_0 = \frac{f}{\|\boldsymbol{I}\|} = \frac{f}{\|\boldsymbol{J}\|}.$$

Luego se normalizan los vectores:

$$\boldsymbol{i} = \frac{\boldsymbol{I}}{\|\boldsymbol{I}\|}, \qquad \boldsymbol{j} = \frac{\boldsymbol{J}}{\|\boldsymbol{J}\|},$$

y se calcula el tercer eje mediante el producto vectorial:

$$\boldsymbol{k} = \boldsymbol{i} \times \boldsymbol{j}.$$

La matriz de rotación queda:

$$R = \begin{bmatrix} \boldsymbol{i}^T \\ \boldsymbol{j}^T \\ \boldsymbol{k}^T \end{bmatrix}.$$

La traslación se obtiene a partir de la imagen del punto de referencia:

$$X_0 = \frac{Z_0}{f} x_0, \qquad Y_0 = \frac{Z_0}{f} y_0,$$

por lo tanto

$$T = \begin{bmatrix} X_0 \\ Y_0 \\ Z_0 \end{bmatrix} = \begin{bmatrix} \frac{Z_0}{f} x_0 \\ \frac{Z_0}{f} y_0 \\ Z_0 \end{bmatrix}.$$

Si las coordenadas de imagen (u_i, v_i) no están centradas, se debe reemplazar

$$x_i \leftarrow u_i - c_u, \qquad y_i \leftarrow v_i - c_v.$$

Iteración POSIT

El algoritmo completo se inicia con

$$\varepsilon_i^{(0)} = 0,$$

lo que equivale a asumir inicialmente una proyección ortográfica escalada.

En la iteración k:

1. Se construyen $\boldsymbol{b}^{(u)}$ y $\boldsymbol{b}^{(v)}$ usando los valores actuales de ε_i.
2. Se resuelven los sistemas lineales

$$A\boldsymbol{I} = \boldsymbol{b}^{(u)}, \qquad A\boldsymbol{J} = \boldsymbol{b}^{(v)}.$$

3. En el caso coplanar, se calculan las dos posibles soluciones mediante las restricciones de ortogonalidad e igual norma.
4. Para cada solución se recuperan $\mathbf{R}$ y $\mathbf{t}$.
5. Se calcula

$$\varepsilon_i^{(k+1)} = \frac{1}{Z_0} \boldsymbol{a}_i^T \boldsymbol{k}.$$

6. Se evalúa la convergencia mediante

$$\max_i \left| \varepsilon_i^{(k+1)} - \varepsilon_i^{(k)} \right| < \tau.$$

Si la condición anterior se cumple, el algoritmo se detiene. En caso contrario, se repite el procedimiento.

Selección de soluciones y factibilidad

En el caso coplanar, POSIT puede producir dos soluciones. Una solución se considera factible si todos los puntos quedan delante de la cámara:

$$Z_i > 0 \qquad \text{para todo } i.$$

Cuando ambas soluciones son factibles, se compara su calidad usando el error de reproyección. Si $\hat{m}_i$ es el punto reproyectado desde la pose estimada, el error promedio puede escribirse como

$$E = \frac{1}{n} \sum_{i=1}^{n} \|m_i - \hat{m}_i\|_2.$$

También puede usarse el RMSE:

$$E_{\text{RMSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \|m_i - \hat{m}_i\|_2^2}.$$

La pose con menor error de reproyección se considera la mejor estimación, aunque en escenas lejanas o con poco efecto de perspectiva pueden mantenerse dos poses plausibles.

Comentario final

POSIT convierte un problema originalmente no lineal de pose en una sucesión de problemas lineales. En cada iteración, los efectos de perspectiva se corrigen mediante los factores ε_i. Para puntos coplanares, el método no fuerza una única solución: puede producir dos poses candidatas, lo cual es importante porque, bajo proyección ortográfica escalada, dos configuraciones simétricas pueden explicar la misma imagen. La decisión final se basa en factibilidad geométrica y error de reproyección.

Referencias

[1] Richard I Hartley. In defense of the eight-point algorithm. *IEEE Transactions on pattern analysis and machine intelligence*, 19(6):580–593, 1997.

[2] Sedam Lee, Daeil Jo, and Yongjin James Kwon. Camera-based automatic landing of drones using artificial intelligence image recognition. 2022.

[3] Denis Oberkampf, Daniel F DeMenthon, and Larry S Davis. Iterative pose estimation using coplanar feature points. *Computer Vision and Image Understanding*, 63(3):495–511, 1996.

[4] Carlos Ricolfe Viala and Antonio José Sánchez Salmerón. Procedimiento completo para el calibrado de cámaras utilizando una plantilla plana. *Revista Iberoamericana de Automática e Informática Industrial RIAI*, 5(1):93–101, 2008.